

Technical Report: Automated Plot Price Prediction & Deployment System

Ali Ahmad Jan

April 7, 2026

System: Python-based Real Estate Prediction System
Focus: Islamabad & Rawalpindi Property Market (2020-2026)

1 Executive Summary

This report details the operation of the `prediction_deployment_system.py` script. The script implements an end-to-end machine learning pipeline that trains a predictive model on historical real estate and macroeconomic data, serializes the model for deployment, and provides an interactive Command-Line Interface (CLI) for users to generate future price forecasts.

2 System Architecture Overview

The system operates in four sequential phases:

1. **Model Training:** Ingests data, engineers features, and trains an XG-Boost Regressor.
2. **Model Serialization:** Saves the trained model, scalers, and encoders into a binary file (pickle).
3. **Model Loading:** Restores the serialized components into memory for rapid inference.
4. **Interactive Interface:** Accepts user inputs, processes them, and displays predictions with confidence intervals.

3 Detailed Process Steps

3.1 Phase 1: Data Ingestion and Preprocessing

The script begins by preparing the training dataset.

- **Data Loading:** It reads two CSV files:
 - `plot_price_historical_dataset.csv`: Contains historical property records (Price, Size, Society, etc.)
 - `pakistan_macroeconomic_indicators_2020_2026.csv`: Contains economic indicators (Policy Rate, Inflation, GDP)
- **Data Merging:** The two datasets are joined on **Year** and **Month** to align economic conditions with specific property transactions.
- **Cleaning:** Missing values are handled using forward-fill (ffill) and backward-fill (bfill).
- **Size Normalization:** The text-based **Size** column (e.g., "10 Marla", "1 Kanal") is converted into a uniform numeric float (e.g., 10.0, 20.0).

3.2 Phase 2: Feature Engineering

The system transforms raw data into a format suitable for machine learning (Time-Series Feature Engineering).

- **Categorical Encoding:** Text labels are converted to integers using `LabelEncoder`.
 - Examples: "Islamabad" $\rightarrow$ 0, "DHA Islamabad" $\rightarrow$ 5
- **Lag Features (Time-Series):** To capture historical trends, the system creates lagged price variables (e.g., `Price_Lag1`, `Price_Lag2`, `Price_Lag3`). These represent the price of the property 1, 2, and 3 months ago.
- **Rolling Statistics:** Moving averages (`Price_MA2`, `Price_MA3`) are calculated to smooth out short-term volatility.
- **Derived Economic Features:**
 - `Real_Rate`: Calculated as `Policy_Rate - Inflation`
 - `Economic_Stress`: A calculated index based on the Current Account Deficit and GDP Growth

3.3 Phase 3: Model Training and Serialization

- **Scaling:** All 18 input features are standardized using `StandardScaler` to have zero mean and unit variance, which is crucial for XGBoost performance.
- **Splitting:** The data is split temporally (85% training, 15% testing) to prevent future data leakage.
- **Training:** An `XGBRegressor` (Extreme Gradient Boosting) is trained on the scaled data.

- **Serialization (Pickling):** The trained model, along with the scaler and label encoders, is saved into a file named `best_model_xgboost.pkl`. This creates a persistent "snapshot" of the intelligence that can be loaded instantly in the future without retraining.

3.4 Phase 4: Prediction Logic (Inference)

The core prediction function `predict_price` operates as follows:

- **Input Extraction:** It accepts a dictionary containing:
 - Property details (City, Society, Size)
 - Economic indicators (Policy Rate, Inflation, etc.)
 - Historical Prices: The user must provide the price history (Lag 1, 2, 3)
- **Encoding:** It uses the loaded `LabelEncoders` to transform text inputs (City, Society) into the numeric codes expected by the model.
- **Vector Construction:** It assembles the 18 input features into a single array in the exact order used during training.
- **Scaling:** It applies the loaded `StandardScaler` to transform the new input data.
- **Prediction:** The `best_model.predict()` function generates the estimated price.
- **Confidence Interval:** It calculates a range ($\pm 10\%$ of the prediction) to provide the user with a margin of error.

3.5 Phase 5: User Interface (CLI)

The script provides a `PropertyPredictionSystem` class to handle user interaction.

- **Interactive Menu:** A command-line loop allows the user to select options (New Prediction, View History, Exit).
- **Input Collection:** The system prompts the user to select cities, societies, and sizes from lists derived from the training data.
- **Data Entry:** It manually requests current economic indicators and, crucially, historical prices (Price 1 quarter ago, etc.).
- **History Management:** Every successful prediction is appended to `prediction_history.json`, creating a permanent log of the system's usage.

Component	Description
XGBoost Regressor	The core algorithm chosen for its ability to handle tabular data and non-linear relationships between macro factors and prices.
StandardScaler	Ensures features like "Policy Rate (~ 10)" and "Price (millions)" are on comparable scales.
LabelEncoder	Enables the model to process categorical text data (Society names) by mapping them to integers.
Pickle (Serialization)	Allows the complex model object to be saved to disk and restored, decoupling the training phase from the deployment phase.

Table 1: Key Technical Components

4 Key Technical Components

5 Usage Requirements & Output

5.1 Input Requirements

To generate a prediction, the user must provide the specific historical prices of the plot (Lag 1, Lag 2, Lag 3) and current macroeconomic indicators. This implies the system is designed for users who have access to detailed historical transaction data or are simulating scenarios.

5.2 Output

- **Visual Console Output:** A formatted text display showing the Estimated Price, Confidence Range, and input parameters.
- **Persistent Storage:**
 - `best_model_xgboost.pkl`: The trained AI model
 - `prediction_history.json`: A log of all predictions made during the session

6 Conclusion

This script serves as a comprehensive standalone tool. It bridges the gap between data science (training/feature engineering) and practical application (user interface/deployment). By implementing a pickle-based architecture, it ensures that the heavy computation of model training is a one-time cost, allowing for fast, real-time predictions in the deployment phase.

End of Report